SIIT CSS326 DATABASE PROGRAMMING LAB

PawStay: Pet Hotel System

A reservation and daily care management platform

Mr. Nattapat Ungphakorn
6722771522 • Frontend & UI/UX

Mr. Kittiphat Khumsawad
6722774252 • DB Design & Backend

Project Description

Existing Operational Bottlenecks

Traditional pet boarding facilities often rely on manual paper ledgers or unstructured messaging apps (e.g., LINE chat), introducing severe business risks:

- Frequent double-booking and room assignment collisions during peak holiday periods.
- Calculation errors when multiple pets from the same owner share a single boarding suite.
- Lack of real-time visibility for owners regarding medication, meals, and bathroom activity logs.

The PawStay Solution

PawStay provides a modern web platform integrating automated database controls to streamline pet lodging and grooming workflows:

- Structured 3-tier Role-Based Access Control (Pet Owner, Staff, and Developer).
- Automated event-driven room status synchronization via MySQL triggers.
- Weight-tiered grooming price calculation handled directly by a MySQL Stored Procedure.

Users and Roles

CLIENT ACCESS

User (Pet Owner)

Registers personal profile and manages registered pets (species, breed, weight, medical conditions). Books available suites with multi-pet allocation and tracks real-time daily care activity feeds.

FRONT DESK ACCESS

Administrator (Staff)

Monitors floor room occupancy grids in real time. Receives high-risk behavioral safety alerts for aggressive animals, logs care routines (meals, walks), and executes automated checkout billing.

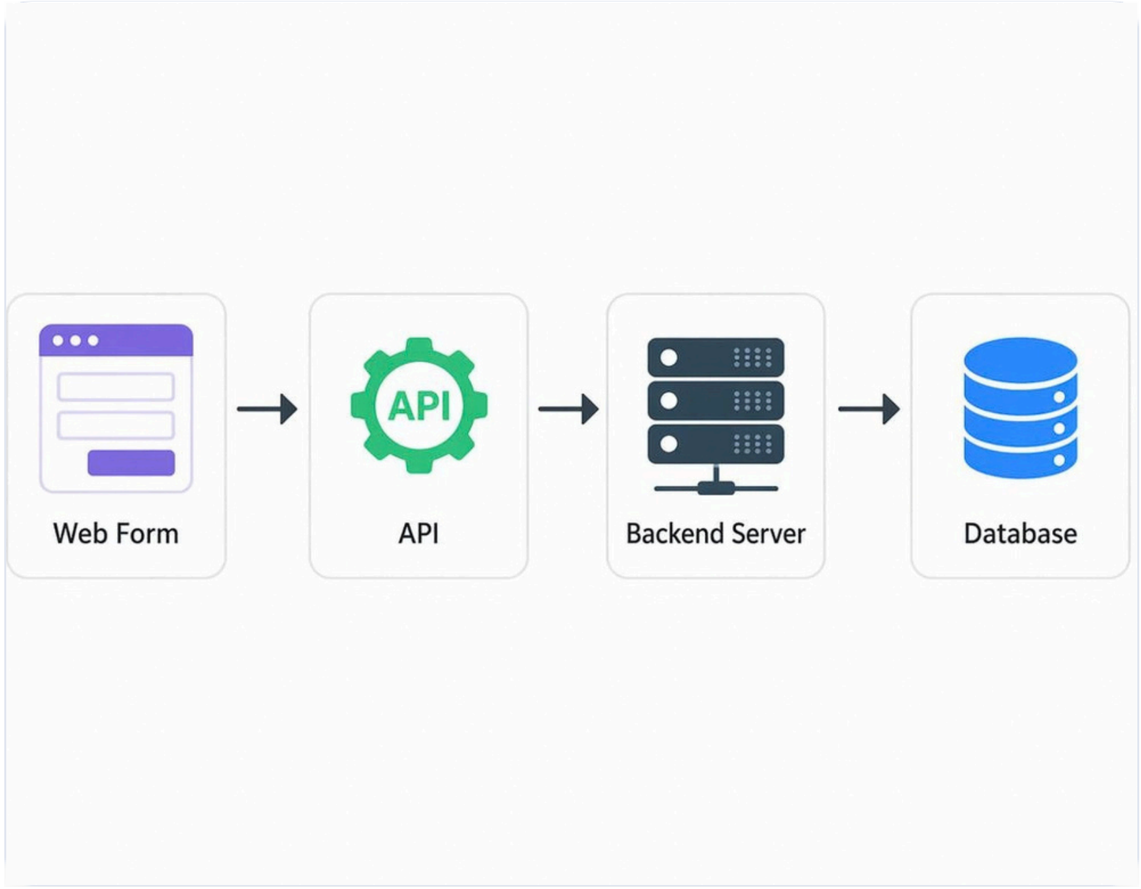
SYSTEM ACCESS

Developer Console

Manages user privilege levels, monitors MySQL connection pool metrics (active vs. idle connections), inspects query audit logs, and directly verifies backend Stored Procedures and Triggers.

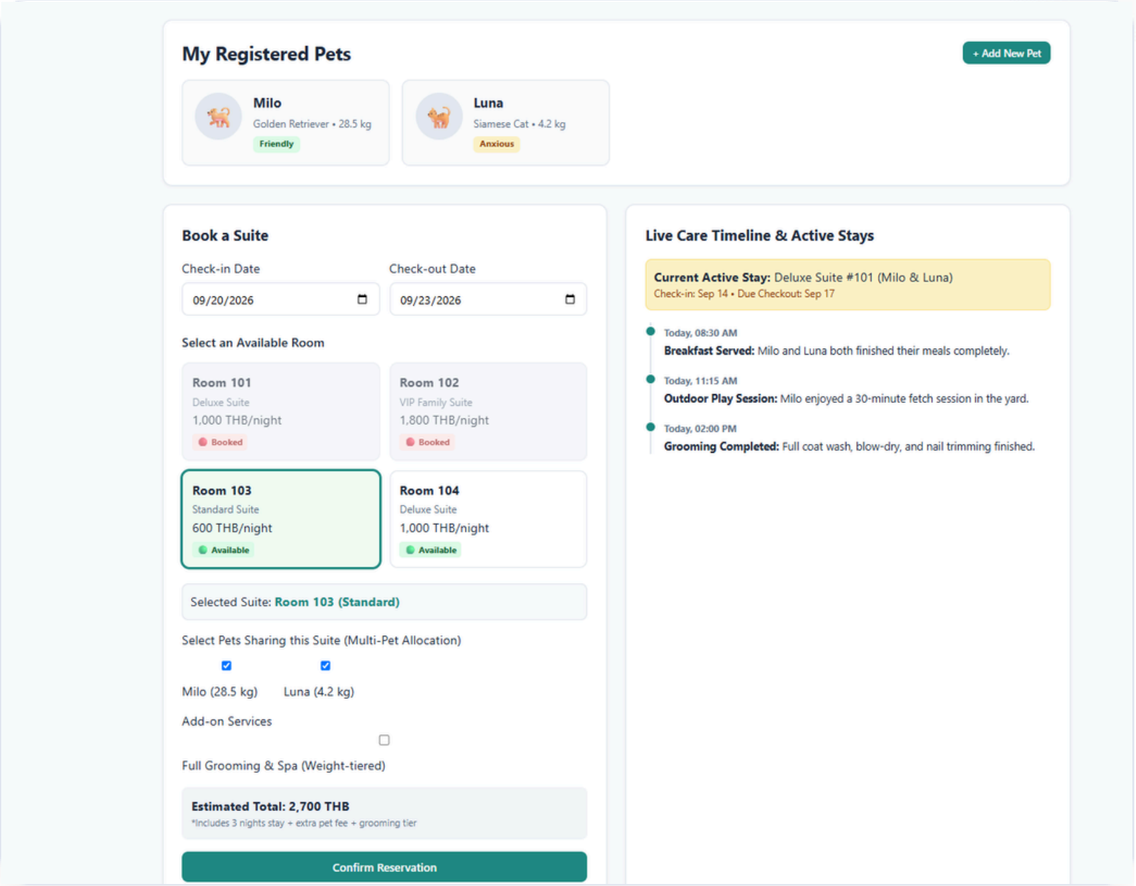
Operational System User Flows

- **User Booking Flow:** Pet Owner selects stay dates and available suite → Server validates room capacity → Database executes parameterized `INSERT` into `bookings` and `booking_pets` .
- **Admin Care Logging:** Staff selects an active stay → Enters meal or medication observations → Server validates input → Database executes `INSERT` into `daily_logs` .
- **Automated Checkout:** Staff clicks checkout → Server calls Stored Procedure `sp_finalize_and_calculate_bill` → Room state automatically flips back to 'Available' via MySQL Trigger.



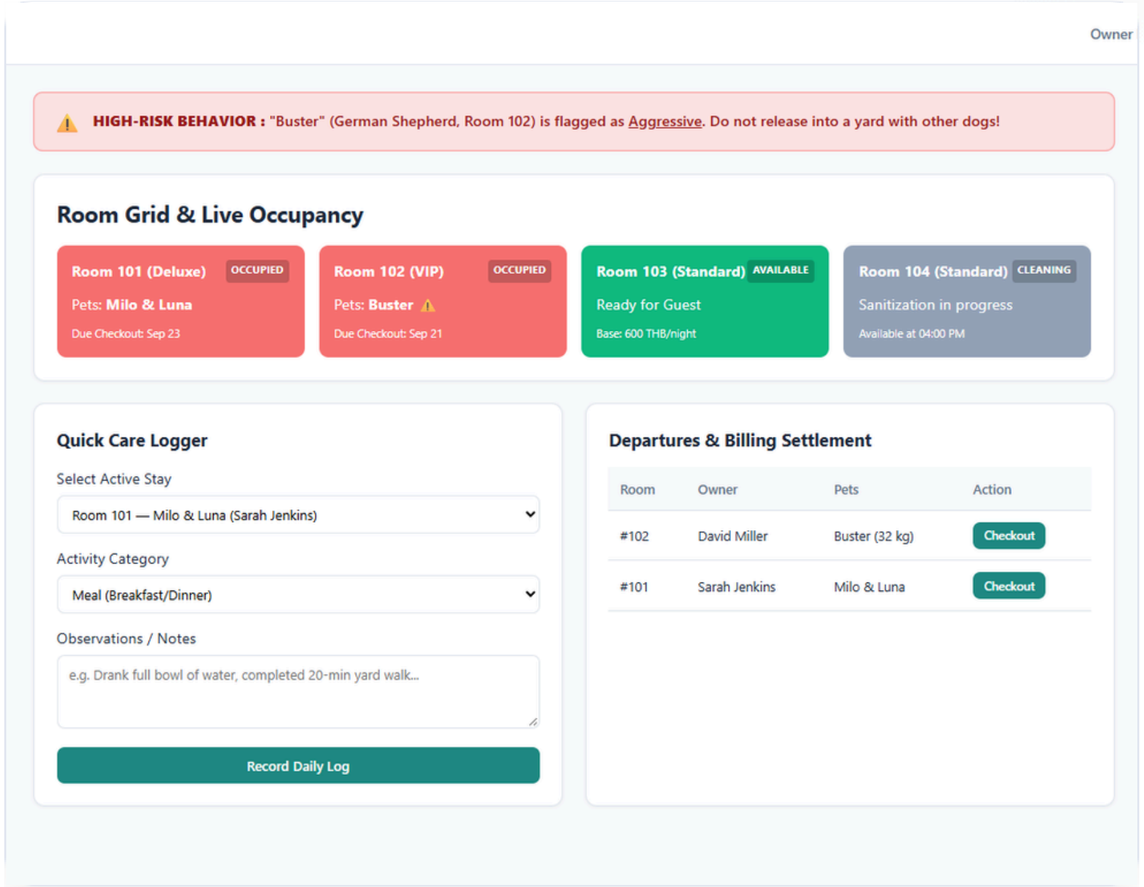
Customer Portal: Page Mockup

- **Pet Profile Manager:** Interactive cards displaying registered dogs and cats, weight, behavior status, and medical history with modal edit controls.
- **Dynamic Room Selector:** Live visual grid displaying suite availability (Room 101-104), nightly base pricing, and multi-pet allocation checkboxes.
- **Live Care Timeline:** Time-stamped event feed showing feeding times, outdoor play sessions, and completed grooming updates for the active reservation.



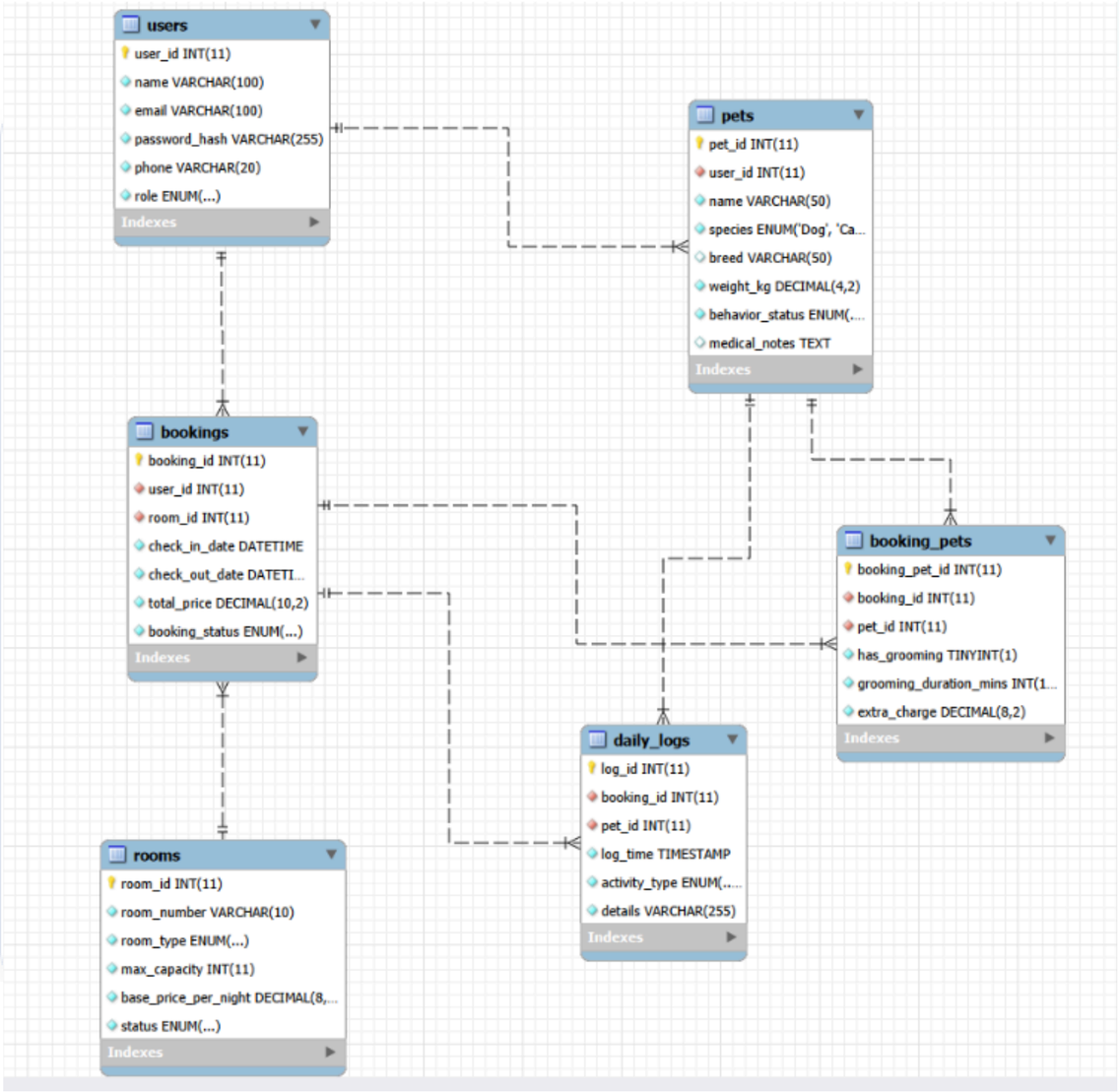
Staff Dashboard: Page Mockup

- ⚠️ **Safety Alert Banner:** Real-time warning strip immediately alerting staff when pets flagged as `Aggressive` are checked in, preventing playgroup mixing.
- ✔️ **Live Room Occupancy Grid:** Color-coded visual floor plan displaying Green (Available), Red (Occupied), and Grey (Maintenance) suite statuses.
- ✔️ **Departures & Settlement:** One-click checkout trigger executing the billing Stored Procedure and releasing room inventory automatically.



Database Design

- **Normalized 3NF Design:** Six interconnected entities eliminating data redundancy and update anomalies across customer and operational data.
- **M:N Relationship Resolution:** `booking_pets` acts as a junction table resolving multi-pet room boarding with dedicated grooming add-on attributes.
- **Integrity Constraints:** Foreign key references enforce `ON DELETE RESTRICT` on customer profiles and `ON DELETE CASCADE` on child log entries



Physical Design

Table	Primary Key	Foreign Keys (References)	Key Columns & Data Types	Constraints & Defaults
users	`user_id INT`	None	email VARCHAR(100), password_hash VARCHAR(255)	UNIQUE(email), ENUM('User','Admin','Dev')
pets	`pet_id INT`	user_id → users	weight_kg DECIMAL(4,2), species ENUM	ON DELETE RESTRICT, behavior_status
rooms	`room_id INT`	None	room_number VARCHAR(10), base_price DECIMAL(8,2)	UNIQUE(room_number), status ENUM
bookings	`booking_id INT`	user_id, room_id	check_in_date DATETIME, total_price DECIMAL(10,2)	booking_status ENUM('Pending','Confirmed',...)
booking_pets	`booking_pet_id INT`	booking_id, pet_id	has_grooming BOOLEAN, extra_charge DECIMAL(8,2)	UNIQUE(booking_id, pet_id), ON DELETE CASCADE
daily_logs	`log_id INT`	booking_id, pet_id	activity_type ENUM, details VARCHAR(255)	DEFAULT CURRENT_TIMESTAMP

CRUD Plan

Operation	Target Page	Target Table(s)	Action Description (User & Application Context)
CREATE (INSERT)	index.html	`pets` `bookings`	Owner registers a new pet or books a suite with multi-pet selection and grooming add-ons.
CREATE (INSERT)	admin.html	`daily_logs`	Staff logs feeding schedules, bathroom breaks, or medical observations for active guests.
READ (SELECT)	index.html	`rooms` `daily_logs`	Owner browses available suites dynamically and monitors live care timelines on their mobile portal.
UPDATE (UPDATE)	admin.html	`bookings` `rooms`	Staff executes checkout; system runs billing Stored Procedure and resets room status to 'Available'.
DELETE (DELETE)	index.html	`bookings` `pets`	Owner cancels upcoming unconfirmed bookings or deletes pet records with no active stay history.

Technology, Stored Procedure, and Trigger

Server-Side MySQL Routines

Stored Procedure: `sp_finalize_and_calculate_bill(IN p_booking_id INT)`

- Calculates night count using `DATEDIFF(check_out, check_in)` .
- Applies multi-pet surcharge (+300 THB/night per extra pet).
- Evaluates weight tiers (<8kg: 500B, 8-20kg: 800B, >20kg: 1,200B) for grooming and updates `total_price` .

Trigger: `trg_after_booking_status_change` automatically locks suite to 'Occupied' when confirmed and unlocks to 'Available' upon completion.

Security & Connection Pooling

bcrypt Hashing & Parameterization:

- User credentials salted and hashed via bcrypt (10 rounds) before persistence; no plaintext passwords stored.
- Strict parameterized queries (`?` placeholders) via `pool.execute()` to eliminate SQL Injection risks.

Express & MySQL Architecture:

- Follows Lab 6 MVC pattern with a shared `mysql2/promise` connection pool (`connectionLimit: 10`).

MVC Pattern (Model-View-Controller)

- *Model: Handles the database structure and operations (db.js, MySQL tables).*
- *View: Provides the user interface and presentation layer (index.html, admin.html).*
- *Controller: Acts as an intermediary that handles logic, processes data, and connects requests from Routes to the Model (controllers/, routes/).*

Week-by-Week Timeline

Weeks 7 – 8

Proposal & Schema

Deliver proposal document and slides. Initialize MySQL physical schema and setup Express pool.

Weeks 9 – 10

Auth & REST APIs

Implement bcrypt user authentication, 3-tier RBAC middleware, and CRUD endpoints for bookings.

Weeks 11 – 12

Automation & UI

Deploy MySQL Stored Procedure and Trigger. Connect HTML/JS front-end interfaces via fetch().

Weeks 13 – 14

Testing & Demo

Consultation Lab 12, end-to-end CRUD verification, and Final Live Project Demonstration (Nov 6).

END OF PROPOSAL PRESENTATION

THANK YOU